

Quadrilateral Hull 판별을 통한 효율적인 2차원 Convex Hull 전처리 알고리즘

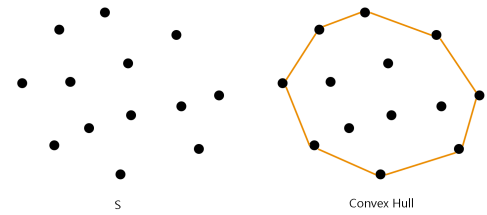
(Abstract)

Convex Hull은 점 집합으로 주어진 영역을 포함하는 가장 작은 볼록 집합으로 산업에서 다양하게 이용되고 있다.

이 논문에서는 주어진 점 집합에서 Convex Hull의 순서 있는 최외곽 점들을 구할 때, Convex Hull의 내부 점들을 미리 배제하여 점 집합의 크기를 축소하는 전처리 알고리즘을 소개한다. 초기 점 집합의 Convex Hull 최외곽들을 구하는데 걸리는 시간보다 전처리 과정을 수행하여 점 집합의 크기를 줄이고 축소된 점 집합의 Convex Hull 최외곽 점들을 구하는데 걸리는 시간이 더 적다. 실제로 알고리즘을 구현하여 실험해 본 결과, 전처리 과정에서의 점 집합 배제율이 95% 이상이었고 점 집합의 밀도가 높아짐에 따라 100%에 근접한 수치를 보였다. 결과적으로 좌표가 특정 범위에 있는 점 집합에 대해 전처리 알고리즘을 적용했을 때 최대 10배 빠르게 Convex Hull을 구할 수 있었다. 이 알고리즘은 점 집합의 좌표 범위 w 에 대해 $O(w^2)$ 의 시간 복잡도를 가지므로 특정 범위에서의 Convex Hull을 효율적으로 구하는데 적합하다.

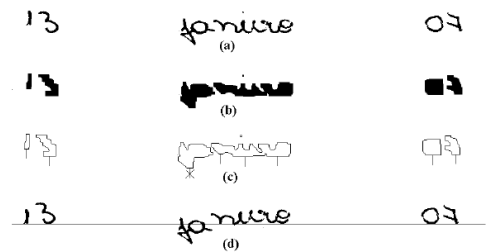
1. 서론

- Convex Hull에 관한 설명
 Convex Hull이란 일반적으로 어떤 점집합 C 에 대해 C 를 포함하는 최소의 볼록 점 집합이다. 하지만 우리의 논문에서는 Convex Hull을 기존과 다르게 재정의할 것이다. 우리는 유한한 크기의 점집합에 대해서만 고려를 할 것이므로 이 경우 Convex Hull은 항상 볼록 다각형이 된다. 그리고 우리는 앞으로 이 볼록 다각형의 순서 있는 꼭짓점들을 Convex Hull이라고 부를 것이다. 앞으로 별다른 설명 없이도 n 은 Convex Hull을 구할 점집합의 크기를, R 은 점집합 원소의 좌표 범위를 나타낸다. ($1 \leq x, y \leq R$)



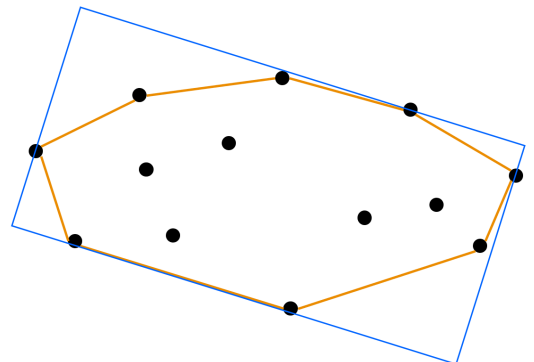
<그림 1. 점집합 S와 S의 Convex Hull>

- Convex Hull이 쓰이는 예
 1) 글자를 인식할 때 Convex Hull 알고리즘이 사용된다. 이는 손글씨의 외곽선에서 Convex 한 부분을 추출하여 글자를 다각형으로 모델링하는 것이다. 글씨의 외곽선에서 Convex



<그림 2. Convex Hull을 이용한 글자 인식>

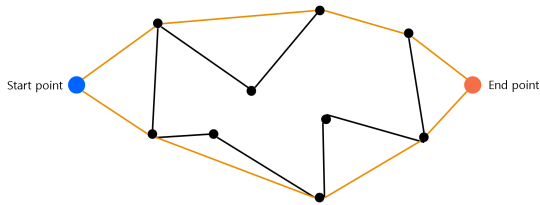
- 2) 물체를 포함하는 최소 넓이의 직사각형을 구하는 문제의 전처리 과정에서 물체의 Convex Hull을 구한다. 물체의 Convex Hull을 다각형으로 근사하면, 물체를 포함하는 최소 넓이의 직사각형은 물체의 Convex Hull을 포함해야 한다. 따라서 그 직사각형은 Convex Hull 다각형의 한 변을 그 직사각형의 한 변으로 공유한다. 이 아이디어를 이용한다면 물체들의 Convex Hull 다각형을 구해 물체를 덮는 최소 넓이의 직사각형을 빠르게 구해낼 수 있다.



<그림 2. Convex Hull을 이용한 물체를 포함하는 최소 직사각형 구하기>

- 3) 현실세계에서 두 위치 간 최단 경로를 구할 때, Convex Hull을 이용할 수 있다.

일반적으로 시작점과 도착점 사이에 넘어갈 수 없는 장애물이 있는 경우, 시작점과 끝점, 그리고 장애물의 Convex Hull을 따라 그린 선이 최단경로이다. 즉 Convex Hull을 구하면 최단경로를 구할 수 있다.



<그림3. Convex Hull을 이용한 최단거리>

2. 배경

- Convex Hull 알고리즘의 발전
1970년 찬드와 카퍼, 1973년 자비스에 의해 Jarvis march라는 Convex Hull을 구하는 알고리즘이 제안되었다. 이후 1972년에는 Graham이 Graham scan이라는 $O(n \log n)$ 시간복잡도를 가지는 알고리즘을 만들었다. 그 이후 Quick Hull, Divide and Conquer 방법을 비롯한 다양한 Convex Hull 알고리즘들이 개발되었다.

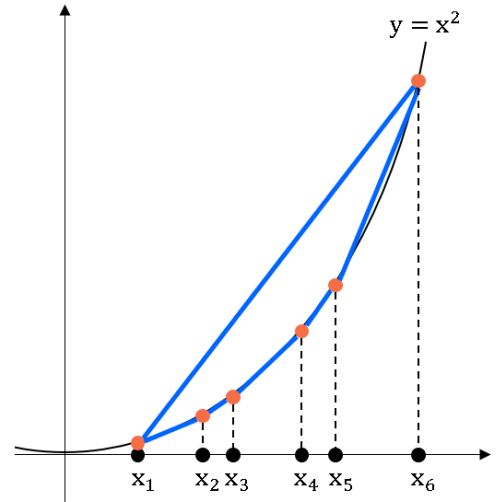
알고리즘	Graham Scan	Monotone Chain	Jarvis March	Quick Hull	Divide and Conquer
제안자	Graham	Andrew	Jarvis	Barber, Dobkin, Huhdanpaa	Unknown
시간복잡도	$O(n \log n)$	$O(n)$	$O(nh)$	$O(n \log h)$	$O(n \log n)$
특정 조건에서의 시간복잡도	$O(n)$ -가장 왼쪽 아래 점을 기준으로 시계방향으로 정렬되어 있을 경우	$O(n)$ -x좌표, y좌표 기준으로 정렬되어 있을 경우			

<표 1. 여러 종류의 Convex Hull 알고리즘>

*점의 개수는 n , Convex Hull에 속하는 점의 개수는 h

- Convex Hull의 시간복잡도
초기에 Jarvis가 제시한 Jarvis march 알고리즘의 시간복잡도는 $O(nh)$ 이다. 하지만 이후에 Graham scan부터 $O(n \log n)$ 의 시간복잡도를 가지는 알고리즘을 제시하였다. 이러한 Convex Hull 알고리즘들의 시간복잡도의 하한은 이미 증명이 되었다. 만약 $x_1, x_2, \dots, x_n$ 을 정렬한다고 할 때, 각 원소들을 모두 $(x_1, x_1^2), (x_2, x_2^2), \dots, (x_n, x_n^2)$ 로 바꾼다. 변형된 점들을 좌표평면에 표시한다면 $y=x^2$ 이차곡선 위에 올라가게 될 것이다. 그리고 이 점들의 Convex Hull을 구한다면 Convex Hull은 x좌표 크기 순서대로 정렬된 점들 일것이다. 이러한 점들을 다시 원래의 숫자로 되돌리는 것에는 선형 시간이 걸린다. 즉, Convex Hull을 구하는 데 걸리는 시간과 점들을 실수로

변형하는 데 걸리는 시간의 합인 시간복잡도로 점들을 정렬할 수 있다는 것이다.



<그림 4. 수 정렬을 Convex Hull로 수행>

하지만 선행된 연구에서 비교 정렬을 하는 데 걸리는 시간복잡도의 하한은 오메가($n \log n$)이었다. 즉, Convex Hull을 구하는 과정의 시간복잡도가 $o(n \log n)$ 이라면 정렬 알고리즘 시간복잡도의 하한에 모순이 발생한다. 따라서 결국 Convex Hull알고리즘의 시간복잡도의 하한도 $n \log n$ 이다. 우리는 기존의 Convex Hull알고리즘의 시간적 한계를 느끼고 시간복잡도가 $o(n \log n)$ 인 알고리즘을 제안해보고자 하였다.

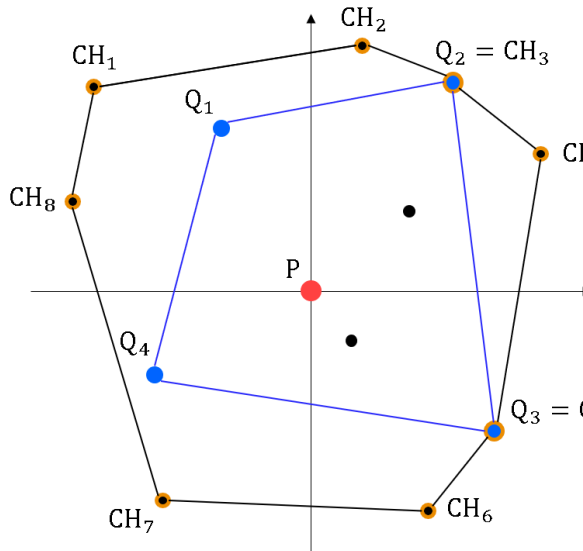
3. 알고리즘의 제안과 구현

- 우리는 먼저 시간복잡도 세타(n)을 가지는 CONVEX HULL 알고리즘을 생각해보았다. 하지만 시간복잡도 n 인 CONVEX HULL 알고리즘을 떠올리는 것은 쉽지 않았고 대신 CONVEX HULL 전처리 과정을 만들고자 하였다. 그리고 우리는 각 점이 특정 조건을 만족할 때, CONVEX HULL의 POINT가 될 수 없다는 것을 발견하게 되었다.
- 경험적 배제 아이디어(QUADRILATERAL HULL)
주어진 점 집합 S 의 점 P 가 CONVEX HULL에 포함되는지는 그 점 사방의 점의 유무에 따라 결정된다. P 를 원점으로 하는 직교 좌표계를 생각하자. 만약 모든 사분면에 점이 1개 이상 존재한다면 각 사분면에 있는 점 1개씩을 꼭짓점으로 하며, P 를 포함하는 사각형을 만들 수 있다. 우리는 그 사각형을 QUADRILATERAL HULL이라고 정의한다. 점 P 의

QUADRILATERAL HULL이 존재한다면 P는 S의 CONVEX HULL에 포함되지 않는다. 하지만 QUADRILATERAL HULL이 만들어지지 않는 상황, 즉, 전체 사분면 중 어느 한 곳이라도 다른 점이 존재하지 않는다면 중심의 점 P는 CONVEX HULL에 포함될 가능성이 있다. 이를 이용해 S에서 QUADRILATERAL HULL이 존재하는 점들을 제외함으로써 점집합 S를 효과적으로 축소할 수 있다.

- 수학적 증명

어떤 점집합 S의 CONVEX HULL을 CH라 하자. 주어진 점집합 S에 포함되고 QUADRILATERAL HULL이 존재하는 한 점 P를 생각하자. 즉, P는 QUADRILATERAL HULL의 내부에 있다. CH는 QUADRILATERAL HULL을 포함하므로 P가 CH 내부에 있다. 그런데 CH의 모든 점은 CH 다각형 경계에 있으므로 P는 CH에 속하지 않는다.



<그림 1. P를 중심으로 한 직교좌표계에서의 QUADRILATERAL HULL과 CONVEX HULL의 관계>

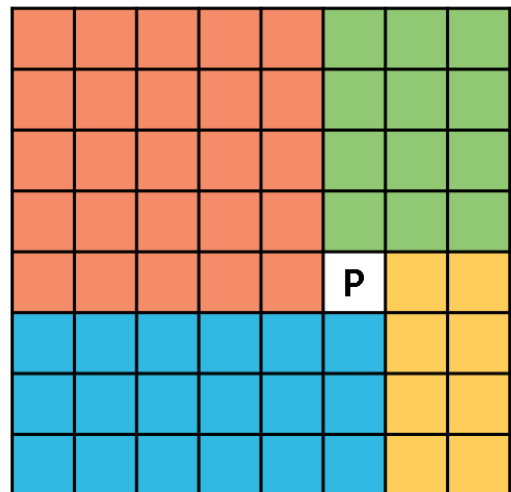
- 알고리즘적 아이디어(구간합)

우리는 임의의 한 점을 중심으로 한 직교좌표계를 정했을 때, 각 사분면에 점이 있는지를 어떻게 판단해야 하는지 고민했다. 만약 각 사분면에 점이 있는지 판별하기 위해 모든 점의 좌표를 확인한다면 최악의 경우 시간복잡도가 $O(N^2)$ 이 되어 효율적이지 못할 것이다. 결국 우리는 사분면에서 점의 유무를 $O(1)$ 의 시간복잡도로 해결해야 모든 점을 탐색하는 시간복잡도가 $O(N)$ 인 알고리즘을 만들어낼 수 있다. 그리고 그러한 방법을 모색하던 중 구간합을 이용하여 구한다는 아이디어를 내게 되었다. 만약 각 사분면의 구간합이 모두 양수라면 이는 모든 사분면에 다른 점이 존재한다는 것이고 결국 기준점의

QUADRILATERAL HULL이 존재하게 되어 CONVEX HULL로부터 기준점을 미리 배제할 수 있게 된다.

- 구현 방식 아이디어

구간합을 다루는 알고리즘의 구현 방식은 다양하게 존재한다. 하지만 완전 탐색으로 구간합을 구하는 방법은 시간복잡도가 $O(N^2)$ 이고 세그먼트 트리와 같은 구간 연산을 빠르게 처리하는 자료구조를 사용하더라도 시간복잡도는 $O(N \log N)$ 으로 기존의 CONVEX HULL 알고리즘과 시간을 비교해 보았을 때 크게 개선된 점이 없다. 하지만 2차원 배열에서의 누적합은 전체 배열은 한번 탐색하는 것으로 구간합을 $O(1)$ 에 구할 수 있도록 해준다. 그래서 우리는 2차원 배열에서의 누적합을 이용하여 우리의 알고리즘을 구현해 보았다. 좌표의 범위를 한정시켰을 때 2차원 배열을 만들 수 있다. 그리고 2차원 배열에 점집합의 점들을 모두 표시하고 모든 인덱스의 누적합을 계산하여 저장한다. 이후 각 점이 CONVEX HULL에서 배제되는지를 판별할 때 누적합의 차를 이용해 빠르게 모든 점들을 확인하는 방법으로 우리의 알고리즘을 구현하였다. 누적합 구현 방식을 사용하여 배열에서 임의의 인덱스 P의 QUADRILATERAL HULL을 판별할 때, P를 기준으로 풍차 형태로 배열을 분할하여 검사를 하는 방법을 사용하기로 했다. 풍차 형태가 아닌 다른 방식으로 사분면을 표현한다면 점 P의 QUADRILATERAL HULL의 존재성을 판별하는데 있어 예외인 경우가 발생할 수 있다.



<그림 2. 인덱스 P를 기준으로 풍차 형태로 사분면을 나누는 모습>

- 구현 방법

$S[x][y] = \text{SUM}(A[i][j])$ ($1 \leq i \leq x, 1 \leq j \leq y$)이라 정의하자.
 $S[x][y] = A[x][y] + S[x-1][y] + S[x][y-1] - S[x-1][y-1]$ 이므로 모든 i, j 에 대해 $S[i][j]$ 를 i 를 1부터 N , j 를 1부터 M 까지 순회하며 모든 $S[i][j]$ 를 구할 수 있다.
 $A[i][j] \geq 0, x_1 \leq i \leq x_2, y_1 \leq j \leq y_2$ 인 i, j 가 존재하는지 판정하는 것은 $\text{SUM}(A[i][j])$ ($x_1 \leq i \leq x_2, y_1 \leq j \leq y_2$) > 0 으로 가능하다.
 이때 이는 $S[x_2][y_2] - S[x_1-1][y_2] - S[x_2][y_1-1] + S[x_1-1][y_1-1]$ 로 $O(1)$ 에 계산할 수 있다.

- 의사 코드

Quadrilateral Hull Preprocess Algorithm

```

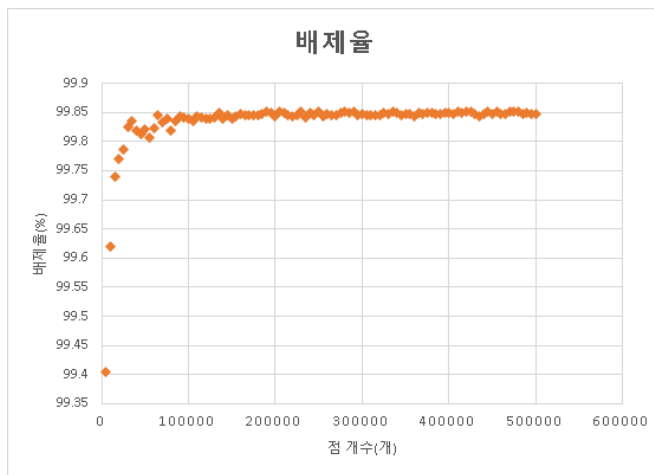
func Preprocess(S: 점집합, R: 좌표 범위  $1 \leq x \leq R, 1 \leq y \leq R$ ){
  // 주어진 점을 좌표 평면에 표시
  for each  $[x, y] \in S$ 
    table[x][y] ← 1;
  // 누적합 배열 만들기
  for i ← 1 to R
    for j ← 1 to R
      S[i][j] ← table[i][j] + S[i-1][j] + S[i][j-1] \
        - S[i-1][j-1];
  //  $x_1 \leq x \leq x_2, y_1 \leq y \leq y_2$ 인  $(x, y)$ 의 개수를 반환하는 함수
  func Sum( $x_1, y_1, x_2, y_2$ ){
    return table[x_2][y_2] - table[x_2][y_1-1] \
      - table[x_1-1][y_2] + table[x_1-1][y_1-1];
  }
  PCHP ← ∅;
  for each  $[x, y] \in S$  {
    if Sum( $x, y+1, R, R$ ) = 0 or
       Sum( $1, y, x-1, R$ ) = 0 or
       Sum( $x+1, 1, R, y$ ) = 0 or
       Sum( $1, 1, x, y-1$ ) = 0
    then PCHP.push_back([x, y]);
  }
}
  
```

4. 실험 및 결과

- 배제율 측정
 우리는 먼저 우리의 전처리 알고리즘을 수행하였을 때, 기존의 점집합에서 점이 얼마나 배제되는지를 실험해 보았다. 실험을 설계할 때, 배열의 범위를 고려할 때, 범위가 너무 작지 않은 경우에는 배열의 범위가 배제율에 미치는 영향을 무시할 수 있다고 판단하였다. 그리고 가로와 세로의 크기가 모두 2000개의 픽셀로 이루어진 평면이라면 우리가 원하는 대부분의 2차원 형태를 표현할 수 있기 때문에 배열의 범위를 2000으로 정하게 되었다. 테스트 케이스를 만들 때는 편재되지 않은 데이터를 얻기 위해 널리 알려진 시간에 대한 랜덤 함수를 이용하여 배열 전체에 점들이 고르게 분포된 데이터를 만들어 실험하였다. 그리고 점의 개수가 너무 적다면 점들이 배제되지 않을 뿐만 아니라 배열에서 고르게 분포하기 힘들고 점의

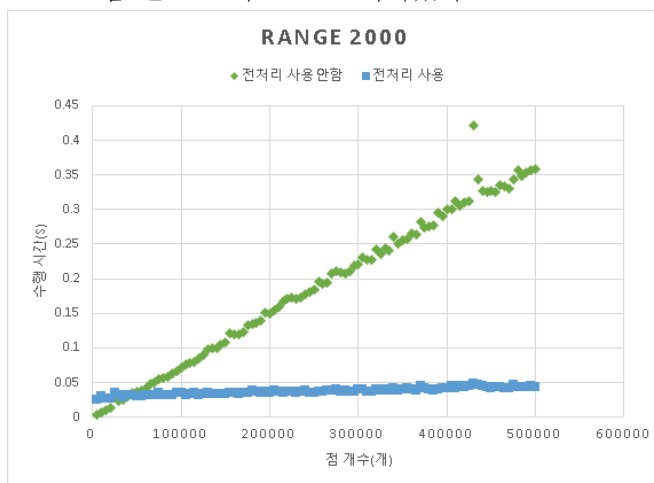
개수가 너무 많다면 전체 배열의 대부분에 점이 채워져서 결국 전체 배열의 CONVEX HULL을 구하는 문제로 변질될 수 있다. 그래서 점의 개수를 적절히 조절하여 5000부터 500000까지 5000단위로 총 100개의 데이터로 실험을 진행하였다.

- 수행 시간 측정
 이후 우리의 전처리 알고리즘의 수행 시간 또한 측정해 보았다. 효율성을 확인하기 위해 기존의 CONVEX HULL 알고리즘인 GRAHAM SCAN과 비교해 보았다. GRAHAM SCAN 알고리즘은 시간복잡도 $O(N \log N)$ 을 가져 CONVEX HULL 알고리즘 중 빠른 편에 속한다. 하나는 전처리 알고리즘을 사용하지 않고 GRAHAM SCAN 알고리즘을 수행한 경우, 다른 하나는 전처리 과정을 수행한 뒤에 GRAHAM SCAN 알고리즘을 수행한 경우의 시간을 비교해보았다. 수행 시간은 누적합 배열의 범위의 제곱에 비례하기 때문에 배제율을 측정할 범위에서 조금 더 큰 범위를 함께 측정해 보아야 한다. 그래서 우리는 배열의 범위가 2000, 3000, 4000인 경우에 각각 실험을 진행하였다. 점집합은 아까와 같이 시간에 대한 랜덤 함수를 이용하여 생성하였고, 점의 개수는 5000부터 500000까지 5000단위로 정하여 총 100개의 데이터로 실험을 진행하였다.
- 실험 결과 분석(배제율)
 주어진 데이터 점 집합에서 전처리 알고리즘을 수행하여 점들이 얼마나 배제되었는지 %단위로 측정하여 그래프로 나타내었다. 가로축은 입력된 점의 개수(개), 세로축은 전처리 과정 시 배제율(%)를 나타낸다. 그래프 분석 결과 점의 개수가 5000개일 때 배제율은 99.4%였고 점의 개수가 30000개일 때부터 배제율이 99.8%를 넘어 점의 개수가 증가함에 따라 배제율은 99.85%에 수렴했다.

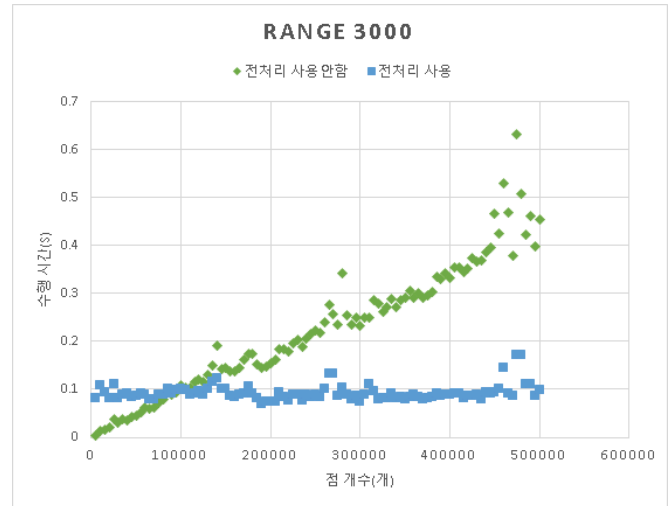


<그래프 1. 전처리 알고리즘 수행 시 입력된 집합에서의 점의 배제율>

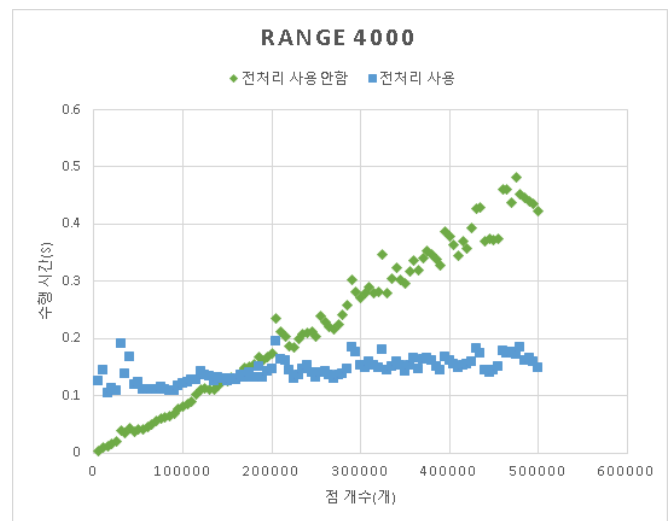
- 실험 결과 분석(수행 시간)
전처리 알고리즘을 사용하지 않고 CONVEX HULL을 구한 경우(녹색)과 전처리 알고리즘을 사용하여 CONVEX HULL을 구한 경우(청색)을 그래프로 나타내었다. 그래프의 가로는 점의 개수(개), 세로는 수행 시간(sec)이다. 그래프 분석 결과(그래프 1, 그래프 2, 그래프 3) 알고리즘 수행시간의 초기값은 전처리 과정을 한 쪽이 더 컸지만 입력받는 점의 개수 증가에 따른 수행시간 증가 비율은 전처리 과정을 수행하지 않은 쪽이 더 컸다. 결국 RANGE가 2000인 경우에는 입력 점 개수가 40000개일 때, RANGE가 3000 일 때는 입력 점 개수가 100000일 때, RANGE가 4000일 때는 점 개수가 150000일 때, 전처리 알고리즘을 사용한 경우와 사용하지 않은 경우의 시간복잡도가 같았다. 그리고 각 상황에서의 점 밀도는 약 0.01로 나타났다.



<그래프 1. RANGE 2000인 경우에서 전처리 과정 유무에 따른 수행 시간 비교>



<그래프 1. RANGE 3000인 경우에서 전처리 과정 유무에 따른 수행 시간 비교>



<그래프 1. RANGE 4000인 경우에서 전처리 과정 유무에 따른 수행 시간 비교>

5. 결론

- 요약
우리는 특정 상황에서 주어진 점집합의 크기를 줄여 Convex Hull을 빠르게 구할 수 있도록 하는 전처리 알고리즘을 제안하고 이를 구현하였다. 우리의 알고리즘은 x, y 좌표가 10000 이하인 자연수인 경우에서 사용할 수 있도록 설계 되었으나, 사용 환경의 공간 제약에 따라 좌표 범위를 확장하거나 축소할 수 있다. 또한 좌표가 음의 정수인 경우에도 모든 좌표가 자연수가 되도록 좌표계를 평행이동하여 알고리즘을 적용할 수 있다. 위 알고리즘을 점집합에 적용했을 경우 점집합의 크기가 1%로 줄어들었다. 우리의 알고리즘은 점집합의 밀도가 높을수록 효율이 올라가며 밀도가

—이상일 때 우리의 알고리즘을 사용한다면 기존의 알고리즘인 Graham scan 알고리즘만을 이용하는 것 보다 빠르게 Convex Hull을 구할 수 있다.

- 시사점
결국 우리의 알고리즘은 기존의 Convex Hull 알고리즘과는 전혀 다른 방법인 누적합을 이용하여 점집합에서 Convex Hull에 포함될 수 없는 점들을 효과적으로 배제하였다. 또한 이 알고리즘의 시간복잡도는 일반적인 Convex Hull 알고리즘의 시간복잡도의 하한보다 작다.
- 기대 효과
사진 및 영상을 처리할 때, 우리의 알고리즘을 사용한다면 더욱 빠르게 Convex Hull을 구할 수 있을 것이다. 이렇게 얻어낸 시간적 이득은 실시간 영상 처리나 대용량 파일을 처리하는 경우 높은 효율을 보여줄 수 있다.
- 발전 가능성
넓은 범위의 자표값을 가지는 점집합의 Convex Hull을 구할 때, 좌표 압축을 이용한다면 충분히 우리의 알고리즘을 이용하여 수행 시간을 줄일 수 있다. 또한 이 알고리즘의 구현 방식을 적절히 변형시킨다면 3D Convex Hull 전처리 알고리즘으로도 발전시킬 수 있다.

[7] 박병주, 이재홍, “평면 점집합에서 정렬을 고려한 개선된 컨벡스 헐 알고리즘”, Journal of IKEEE.Vol.17,No.1,029~035,March 2013
<https://www.koreascience.or.kr/article/JAKO201311637859348.pdf>

[8] S J A Garnés, M E Morita, Jacques Facon, R Sampaio, Flavio Bortolozzi, Robert Sabourin “Correcting handwriting baseline skew using direct methods on the weighted least squares”,January 1999

참고문헌

[1] princi singh/akarshsmani/duang, Convex Hull | Set 1 (Jarvis's Algorithm or Wrapping),
<https://www.geeksforgeeks.org/convex-hull-set-2-graham-scan/?ref=rp>,(2019.07.25)

[2] ‘Graham Scan - Wikipedia’,

https://en.wikipedia.org/wiki/Graham_scan

[3] 홍기천, 오일석, “2차원 패턴의 볼록 헐 알고리즘”, 한국멀티미디어학회논문지, 제 4권 4호, pp.364-365, 2001.8

[4] 차혜경, 신성용, “효율적인 Convex Hull 알고리즘과 그 응용”, 정보과학회지, 제 5권 4호, pp. 59-70

[5] Thaddeus Abiy, Beakal Tiliksew, Josh Silverman, ‘Brilliant’,
<https://brilliant.org/wiki/convex-hull/>

[6] Justicehui, ‘Dynamic Segment Tree’,
<https://justicehui.github.io/medium-algorithm/2020/02/28/Dynamic-Seg/> (2015.10.2)